

O Fim do Autocomplete: De Vibe Coding a Agentic Coding & A Arquitetura de Quatro Camadas: Tela, Harness, LLM e Tools — Um Recorte de AI Driven Development: Do Zero ao Deploy

Heverton Eduardo Peres

RESUMO

Este artigo examina a transição do paradigma de vibe coding para o de agentic coding na engenharia de software contemporânea, e o modelo arquitetural de quatro camadas — Tela, Harness, LLM e Tools — que a literatura técnica converge em descrever como substrato dessa transição. Trata-se de um recorte investigativo documental, derivado do dossiê de pesquisa já indexado para a obra AI Driven Development: Do Zero ao Deploy, sem coleta primária de dados: a metodologia consiste na recuperação seletiva de blocos temáticos do dossiê-mãe por relevância e na síntese analítica dessas fontes segundo o framework ACAD, com citação autor-data (NBR 10520). Os resultados indicam que a autonomia agêntica só se sustenta como prática confiável quando ladeada por controles externos ao modelo — notadamente o TDD como definição prévia de correção — e que a camada Tela (intent preview, approval gates, hybrid autonomy, raio de impacto) e a camada Harness (pipeline determinístico de permissões, gerenciamento de contexto, runtime do agente) operam como as duas metades de um único contrato de confiança entre supervisão humana e execução autônoma. Discutem-se ainda riscos documentados na camada de ferramentas conectadas via protocolo aberto (MCP), como o envenenamento de ferramentas, que reforçam a necessidade de aplicação determinística de regras na camada Harness. Conclui-se que a adoção corporativa de ferramentas agênticas de codificação deve ser avaliada pela robustez do harness que envolve o modelo, não apenas pela capacidade de raciocínio deste.

Palavras-chave: agentic coding, arquitetura de agentes, harness de codificação, vibe coding, governança de IA.

ABSTRACT

This article examines the shift from the vibe coding paradigm to agentic coding in contemporary software engineering, and the four-layer architectural model — Screen, Harness, LLM, and Tools — that the technical literature converges on describing as the structural substrate of that shift. This is a documentary investigative excerpt, derived from the research dossier already indexed for the book AI Driven Development: Do Zero ao Deploy, with no primary data collection: the methodology consists of selective retrieval of thematic blocks from the parent dossier by relevance and analytical synthesis of these sources following the ACAD framework, with author-date citation (NBR 10520). Results indicate that agentic autonomy only holds up

as a reliable practice when flanked by controls external to the model itself — notably TDD as a prior definition of correctness — and that the Screen layer (intent preview, approval gates, hybrid autonomy, blast radius) and the Harness layer (deterministic permission pipeline, context management, agent runtime) operate as the two halves of a single trust contract between human oversight and autonomous execution. Documented risks in the tool layer connected via an open protocol (MCP), such as tool poisoning, are also discussed, reinforcing the need for deterministic rule enforcement at the Harness layer. The article concludes that corporate adoption of agentic coding tools should be evaluated by the robustness of the harness surrounding the model, not merely by the model’s reasoning capability.

Keywords: agentic coding, agent architecture, coding harness, vibe coding, AI governance.

1 INTRODUÇÃO

1.1 Contextualização e Problema de Pesquisa

Entre 2024 e 2026 a engenharia de software atravessa uma mudança estrutural que a literatura técnica compara à adoção do DevOps e do Agile: modelos de linguagem de grande porte (LLMs) deixam de operar como “autocomplete avançado” — paradigma denominado *vibe coding*, em que o desenvolvedor permanece integralmente no loop, revisando cada sugestão em modo conversacional — para atuar como agentes autônomos capazes de planejar, executar, testar e iterar tarefas inteiras do ciclo de engenharia sob supervisão mínima, paradigma denominado *agentic coding* (ARXIV, 2025). A distinção entre os dois paradigmas não é apenas de grau de autonomia, mas de arquitetura de controle: a codificação agêntica trata testes automatizados, linting, integração contínua e revisão de código como a superfície que torna a saída do agente auditável e confiável, ao passo que a codificação por vibe trata esses controles como opcionais, o que eleva o risco operacional e reduz a responsabilização (*accountability*) em produção (ARXIV, 2025; FORRESTER, 2026).

Dados de mercado recentes sustentam a relevância do problema: relatórios de analistas indicam que a maioria das organizações de desenvolvimento de software já utiliza IA de forma ativa em algum ponto do ciclo de vida (FUTURUM GROUP, 2026; FORRESTER, 2026), e fornecedores de plataforma relatam iniciativas de automação de ponta a ponta do *software development lifecycle* (SDLC) apoiadas em agentes (FUJITSU, 2026; MICROSOFT, 2026). Esse movimento de mercado, no entanto, expõe uma lacuna conceitual: a difusão de ferramentas agênticas de codificação (Claude Code, Cursor, GitHub Copilot, entre outras) sem que a arquitetura interna que sustenta a autonomia desses sistemas seja amplamente compreendida por

quem os adota — a diferença entre “ter um LLM” e “ter um agente de codificação” é tratada, na prática corporativa, como incidental, quando na verdade é estrutural (MINDSTUDIO, 2026; AIMULTIPLE, 2026).

Um segundo eixo do problema de pesquisa concerne à tensão entre codificação agêntica e práticas clássicas de engenharia, notadamente o TDD (*Test-Driven Development*). A literatura reporta que agentes de IA geram código plausível em segundos, mas “parecer plausível” e “de fato funcionar” são propriedades distintas: sem *guardrails*, a saída do agente passa no *vibe check* mas falha em produção (ARXIV, 2025). A resposta documentada pela comunidade técnica é reforçar, não abandonar, o TDD — escrever o teste antes de qualquer implementação define o que é “correto” antes que o agente gere uma linha de código, funcionando como camada de controle estrutural externa ao próprio modelo (ARXIV, 2025; ARXIV, 2026).

1.2 Objetivo do Recorte

Este recorte investigativo tem por objetivo caracterizar (i) a transição conceitual de *vibe coding* para *agentic coding* como mudança de paradigma de engenharia, e (ii) o modelo arquitetural de quatro camadas — Tela, Harness, LLM e Tools — que a literatura técnica converge em descrever como o substrato estrutural dessa transição, com ênfase nas camadas Tela e Harness, responsáveis, respectivamente, pela interface de supervisão humana (*intent preview*, *approval gates*, *hybrid autonomy*, estimativa de “raio de impacto”) e pelo runtime que transforma um modelo de linguagem em um agente de codificação capaz (MINDSTUDIO, 2026; PILLITTERI, 2026; WAVESPEED, 2026).

1.3 Justificativa e Delimitação

A justificativa do recorte decorre da constatação, presente na literatura consultada, de que a arquitetura de quatro camadas é tratada de forma dispersa — fornecedores de harness (GITHUB, 2026; MICROSOFT, 2026), fabricantes de modelo (ANTHROPIC, 2026) e analistas de mercado (FORRESTER, 2026; FUTURUM GROUP, 2026) descrevem partes do mesmo fenômeno sob vocabulários distintos, sem uma síntese única que relacione paradigma (*vibe* versus *agentic*), arquitetura (as quatro camadas) e superfície de controle humano (a camada Tela). Este artigo delimita-se aos capítulos 1 a 3 do dossiê-mãe que fundamenta a obra da qual deriva, não abrangendo as camadas LLM e Tools em profundidade técnica de implementação — objeto de recorte posterior — mas tratando-as na medida necessária para situar o papel das camadas Tela e Harness na cadeia de decisão do agente (ANTHROPIC, 2026; AGENTA, 2026; BLAXEL, 2026; COMET, 2026; HARTENFELLER, 2026; IBM, 2026; PROMPTHUB, 2026; RESEARCHGATE, 2026; ARTEZIO, 2026; WIKIPEDIA, 2026; HUMANLAYER, 2026; KONISHI, 2026; OWASP, 2026; ARXIV, 2024).

2 METODOLOGIA

2.1 Natureza do Recorte

Este artigo constitui um recorte investigativo de natureza documental, derivado de um dossiê de pesquisa técnica mais amplo, previamente minerado e indexado para a obra “AI Driven Development: Do Zero ao Deploy”. Não se trata de pesquisa empírica com coleta primária de dados — não há experimento controlado, estudo de caso único nem levantamento com participantes — mas de análise qualitativa de fontes secundárias já reunidas: documentação oficial de fornecedores (ANTHROPIC, 2026; GITHUB, 2026; MICROSOFT, 2026), preprints acadêmicos (ARXIV, 2024; ARXIV, 2025; ARXIV, 2026), relatórios de analistas de mercado (FORRESTER, 2026; FUTURUM GROUP, 2026) e material técnico de blogs especializados (MINDSTUDIO, 2026; AIMULTIPLE, 2026; WAVESPEED, 2026; PILLITTERI, 2026). Essa delimitação metodológica é deliberada: o objetivo do recorte é sintetizar e relacionar achados já disponíveis, não gerar dado novo.

2.2 Procedimento de Reaproveitamento do Dossiê

O dossiê-mãe foi indexado em blocos temáticos por meio de um mecanismo de recuperação por relevância (TF-IDF), permitindo consulta seletiva por termos em vez de carregamento integral do corpus. Para este recorte, a consulta priorizou os blocos correspondentes aos capítulos 1, 2 e 3 do sumário macro do livro-mãe — respectivamente “Fundamentos de AI Driven Development”, “Arquitetura em 4 Camadas: Tela, Harness, LLM, Tools” e material correlato sobre configuração prática de harness. Nenhuma varredura web adicional foi realizada: o critério metodológico central é que toda afirmação factual do artigo remonta a um bloco já presente no dossiê-mãe, nunca a conhecimento não rastreável.

2.3 Critério de Seleção das Fontes

Três critérios guiaram a seleção dos blocos e das referências citadas: (i) pertinência direta a um dos três pilares do recorte — a transição vibe-para-agentic (ARXIV, 2025), a arquitetura de quatro camadas (MINDSTUDIO, 2026; PILLITTERI, 2026) e a relação entre a camada Tela (supervisão humana) e a camada Harness (runtime do agente) (WAVESPEED, 2026; ANTHROPIC, 2026); (ii) atualidade, com preferência por fontes de 2025-2026 e inclusão de preprints anteriores apenas quando descrevem fundamentos ainda vigentes na literatura mais recente (ARXIV, 2024); e (iii) triangulação, isto é, preferência por afirmações corroboradas por mais de uma fonte independente — por exemplo, a caracterização da camada Harness como intermediária entre interface e modelo aparece tanto em material de fornecedor de IDE (GITHUB, 2026) quanto em análise comparativa de mercado (AIMULTIPLE, 2026; WAVESPEED, 2026) e em conteúdo técnico especializado (MINDSTUDIO, 2026; PILLITTERI, 2026).

2.4 Construção Textual e Citação

A redação seguiu o framework ACAD (Contextualização, Referencial Teórico, Análise/Desenvolvimento, Síntese Parcial) dentro de cada seção IMRaD, com citação autor-data (NBR 10520) para toda afirmação factual, métrica ou definição técnica extraída do dossiê. Não há, portanto, um “método experimental” a relatar no sentido das ciências naturais — o objeto deste artigo é uma síntese analítica de literatura técnica e de mercado sobre arquitetura de agentes de codificação, e a “metodologia” descrita aqui é o procedimento editorial-documental que sustenta essa síntese, incluindo o uso de fontes como FUJITSU (2026), RESEARCHGATE (2026), ARTEZIO (2026), IBM (2026), BLAXEL (2026), AGENTA (2026), COMET (2026), PROMPTHUB (2026), HARTENFELLER (2026), HUMANLAYER (2026), KONISHI (2026), WIKIPEDIA (2026) e OWASP (2026) para compor o quadro de referência de suporte às três seções restantes deste artigo.

3 RESULTADOS E DISCUSSÃO

3.1 Do Vibe Coding ao Agentic Coding: Ruptura de Paradigma

A síntese das fontes consultadas converge em tratar a passagem do *vibe coding* para o *agentic coding* como ruptura, não como continuidade incremental. No paradigma anterior, o LLM opera sob o comando “ajude-me a escrever código”: o desenvolvedor formula a intenção, revisa cada trecho gerado e decide, turno a turno, se aceita a sugestão (MINDSTUDIO, 2026). No paradigma agêntico, a relação se inverte para “revise o que eu fiz” — o agente planeja, executa múltiplos passos (leitura de arquivos, edição, execução de comandos, testes) e apresenta um resultado já materializado para aprovação humana posterior (ARXIV, 2025; MINDSTUDIO, 2026). Forrester (2026) descreve esse movimento como parte de uma transição mais ampla de “assistentes de código” para “agentes orquestrados de todo o SDLC”, com adoção corporativa relatada por Futurum Group (2026) e Fujitsu (2026) como já majoritária entre organizações de desenvolvimento de software.

Essa ruptura tem um custo estrutural: quanto maior a autonomia do agente, maior a dependência de controles externos ao próprio modelo para que a saída gerada seja confiável. A literatura converge em apontar o TDD como o principal desses controles no nível de código — não como prática opcional, mas como definição prévia de “correção” que o agente não pode negociar ou contornar (ARXIV, 2025). Frameworks emergentes de TDD orientado a agentes propõem análise de impacto automatizada como camada adicional de verificação antes que uma mudança gerada por IA seja aceita (ARXIV, 2026). O padrão que se repete nas fontes é o mesmo em espírito, ainda que descrito por atores distintos: Microsoft (2026) relata pipelines de ponta a ponta apoiados em Azure e GitHub com portões de verificação automatizados, Github (2026) documenta o comportamento de prompts de sistema que condicionam a autonomia do agente a testes prévios, e Anthropic (2026) recomenda, na literatura sobre agentes efetivos, buscar a

solução mais simples possível e só aumentar a complexidade orquestrada quando estritamente necessário — porque cada grau de autonomia adicional troca latência e custo por desempenho, e essa troca deve ser deliberada.

3.2 A Arquitetura de Quatro Camadas: Tela, Harness, LLM, Tools

A literatura técnica converge para um modelo arquitetural de responsabilidades distintas e contratos bem definidos entre quatro camadas: Tela, Harness, LLM e Tools (Pillitteri, 2026; Wavespeed, 2026; Mindstudio, 2026; Aimultiple, 2026). A camada **Tela** (UI/CLI/IDE) é a superfície de supervisão humana; evoluiu do paradigma “ajude-me a escrever código” para “revise o que eu fiz”, e os padrões de 2026 documentados nas fontes incluem: *intent preview* — resumo do plano antes da execução —, *approval gates* para ações de alto risco, *hybrid autonomy* — decisões de baixo risco automáticas, ações consequentes escaladas ao humano — e estimativa explícita de “raio de impacto” (*blast radius*) antes da aprovação (Mindstudio, 2026). A camada **Harness** (runtime do agente) é onde a arquitetura de quatro camadas se torna operacionalmente concreta: o harness fornece ferramentas, gerenciamento de contexto e o ambiente de execução que transformam um modelo de linguagem em um agente de codificação capaz (Pillitteri, 2026). Github (2026) e Wavespeed (2026) descrevem essa camada como decisora do que é *permitido* — cada ferramenta possui seu próprio portão de permissão que verifica um pipeline de regras antes de qualquer execução —, ao passo que a camada LLM decide o que *tentar*. A camada **LLM** é o raciocínio propriamente dito, e a camada **Tools** é a superfície de efeito real no mundo (leitura/escrita de arquivos, execução de comandos, chamadas a serviços externos) — ambas tratadas neste recorte apenas na medida necessária para situar o papel de Tela e Harness na cadeia de decisão (Anthropic, 2026; Agenta, 2026; Blaxel, 2026).

Frameworks de orquestração como os descritos na literatura sobre agentes efetivos implementam padrões recorrentes: *prompt chaining* — cada chamada de LLM processa a saída da anterior —, *routing* — um LLM classifica a entrada e direciona para uma tarefa especializada —, *parallelization* — chamadas de LLM em paralelo —, *orchestrator-workers* — um LLM central decompõe tarefas e delega a LLMs trabalhadores — e *evaluator-optimizer* — uma chamada de LLM gera uma resposta enquanto outra a avalia, em ciclo (Anthropic, 2026). Esses padrões não substituem a arquitetura de quatro camadas; operam **dentro** da camada LLM, orquestrando como o raciocínio é decomposto antes de acionar a camada Tools através do contrato definido pelo Harness (Comet, 2026; Prompthub, 2026).

3.3 Camada Tela e Camada Harness: Intent Preview e o Runtime do Agente

O ponto de maior densidade analítica deste recorte está na interface entre as camadas Tela e Harness, porque é ali que a autonomia agêntica é negociada com a supervisão humana em tempo real. O *intent preview* — resumo do plano antes da execução — não é apenas um recurso de interface; é o mecanismo pelo qual a camada Tela expõe, de forma legível a humanos, uma decisão que a camada Harness já validou como permitida (Mindstudio, 2026; Aimultiple, 2026). Isso implica que a camada Harness precisa avaliar permissões *antes* que a Tela apresente o

plano, não depois — a ordem de avaliação é parte do contrato entre as duas camadas. Github (2026) documenta esse comportamento em termos de arrays de permissão (`allow`, `deny`, `ask`) associados a padrões de comando, e Konishi (2026) descreve o mesmo runtime em termos de configuração prática de arquivo `settings.json`, que controla qual modelo roda, quais comandos de shell são permitidos, quais servidores MCP se conectam e quais *hooks* disparam em edições de arquivo.

A abordagem de segurança subjacente a esse runtime é descrita na literatura como multicamadas: permissões como camada de aplicação diária, configurações gerenciadas como camada de política corporativa, *hooks* como camada de aplicação determinística, e controles de protocolo de ferramentas (MCP) como camada de governança — a analogia recorrente nas fontes consultadas é tratar o agente de IA “como um novo funcionário júnior com acesso root”: dar apenas o acesso necessário, observar o que ele faz, e checar duas vezes quando tenta algo arriscado (Humanlayer, 2026; Konishi, 2026). Essa analogia condensa, em termos operacionais, o que a camada Tela expõe ao humano (o *intent preview* e o *blast radius*) e o que a camada Harness aplica internamente (o pipeline de regras de permissão) — as duas camadas são, na prática, as duas metades de um único contrato de confiança.

No nível de ferramentas expostas ao agente, o padrão de *tool use* documentado inclui um `input_schema` (JSON Schema); quando o modelo decide usar uma ferramenta, retorna um indicador de razão de parada com um ou mais blocos de chamada de ferramenta, e a aplicação executa a operação e devolve o resultado (Anthropic, 2026; Hartenfeller, 2026). Existem ferramentas executadas na aplicação do usuário (incluindo ferramentas com esquema definido pelo próprio provedor do modelo) e ferramentas executadas na infraestrutura do provedor, como busca e execução de código remotos (Anthropic, 2026; Agenta, 2026). Essa distinção importa para a camada Harness porque determina onde o portão de permissão precisa ser aplicado: ferramentas client-side exigem que o harness intercepte a chamada antes da execução local; ferramentas server-side deslocam parte da responsabilidade de controle para o próprio provedor (Ibm, 2026).

Riscos de segurança documentados para esse runtime concentram-se sobretudo na camada de ferramentas conectadas via protocolo aberto (MCP): a literatura cataloga ataques de “envenenamento de ferramenta” (*tool poisoning*), em que uma descrição de ferramenta maliciosa manipula o comportamento do agente sem que o usuário perceba (Owasp, 2026; Wikipedia, 2026), reforçando por que a camada Harness — e não apenas a camada Tela — precisa ser o ponto de aplicação determinística de regras, já que a interface de supervisão humana não tem visibilidade sobre o conteúdo interno de cada chamada de ferramenta (Researchgate, 2026; Artezio, 2026).

4 CONCLUSÃO

A síntese apresentada neste recorte sustenta que a transição de *vibe coding* para *agentic coding* não é uma mudança de grau de conveniência, mas uma mudança de arquitetura de controle: a

autonomia do agente só se torna confiável quando ladeada por superfícies de verificação explícitas — testes automatizados como definição prévia de correção (Arxiv, 2025), e um modelo de quatro camadas — Tela, Harness, LLM e Tools — que distribui, entre interface de supervisão humana e runtime do agente, a responsabilidade de decidir o que é permitido antes de decidir o que é tentado (Pillitteri, 2026; Mindstudio, 2026). A interseção entre Tela e Harness, examinada com maior densidade neste artigo, mostra que padrões como *intent preview*, *approval gates* e estimativa de “raio de impacto” só funcionam porque o Harness já aplicou, previamente, um pipeline determinístico de permissões (Github, 2026; Konishi, 2026).

Como implicação prática, a literatura converge em recomendar que a adoção corporativa de ferramentas agênticas de codificação seja avaliada não pelo poder de raciocínio do modelo isoladamente, mas pela robustez do harness que o envolve — sua capacidade de expor *intent preview* legível, aplicar portões de aprovação graduados e resistir a ataques direcionados à camada de ferramentas, como o envenenamento de ferramentas via MCP (Owasp, 2026; Anthropic, 2026). Capítulos subsequentes desta série de recortes tratam, com maior profundidade técnica, as camadas LLM e Tools e a configuração prática de harness em ambiente de produção.

REFERÊNCIAS

AGENTA. The guide to structured outputs and function calling with LLMs. 2026. Disponível em: <https://agenta.ai/blog/the-guide-to-structured-outputs-and-function-calling-with-llms>. Acesso em: 02 ago. 2026.

AIMULTIPLE. Top agent harnesses: Claude Code vs Codex. 2026. Disponível em: <https://aimultiple.com/agent-harness>. Acesso em: 02 ago. 2026.

ANTHROPIC. Building effective AI agents. 2026. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ARTEZIO. 2026 playbook for software development — LLMs’ roadmap for languages, skills & AI. 2026. Disponível em: <https://www.artezio.com/pressroom/blog/playbook-development-languages/>. Acesso em: 02 ago. 2026.

ARXIV. Agentic AI in the software development lifecycle. 2026. Disponível em: <https://arxiv.org/pdf/2604.26275>. Acesso em: 02 ago. 2026.

ARXIV. Towards optimizing the costs of LLM usage. 2024. Disponível em: <https://arxiv.org/pdf/2402.01742>. Acesso em: 02 ago. 2026.

ARXIV. Vibe coding vs. agentic coding: fundamentals and practical implications of agentic AI. 2025. Disponível em: <https://arxiv.org/pdf/2505.19443>. Acesso em: 02 ago. 2026.

BLAXEL. What is LLM function calling?. 2026. Disponível em: <https://blaxel.ai/blog/what-is-llm-function-calling>. Acesso em: 02 ago. 2026.

COMET. Prompt engineering for agentic AI systems: an introduction. 2026. Disponível em: <https://www.comet.com/site/blog/prompt-engineering/>. Acesso em: 02 ago. 2026.

FORRESTER. Agentic software development takes the lead: from code assistants to orchestrated SDLC agents. 2026. Disponível em: <https://www.forrester.com/blogs/agentic-software-development-takes-the-lead-from-code-assistants-to-orchestrated-sdlc-agents/>. Acesso em: 02 ago. 2026.

FUJITSU. Fujitsu automates entire software development lifecycle with new AI-driven software development platform. 2026. Disponível em: <https://global.fujitsu/en-global/pr/news/2026/02/17-01>. Acesso em: 02 ago. 2026.

FUTURUM GROUP. AI reaches 97% of software development organizations. 2026. Disponível em: <https://futuresgroup.com/press-release/ai-reaches-97-of-software-development-organizations/>. Acesso em: 02 ago. 2026.

GITHUB. Claude Code system prompts. 2026. Disponível em: <https://github.com/Piebald-AI/claude-code-system-prompts>. Acesso em: 02 ago. 2026.

HARTENFELLER. Best practices for LLM tools or function calling for Oracle developers. 2026. Disponível em: <https://hartenfeller.dev/blog/ai-tools-best-practice-oracle>. Acesso em: 02 ago. 2026.

HUMANLAYER. Writing a good CLAUDE.md. 2026. Disponível em: <https://www.humanlayer.dev/blog/writing-a-good-claude-md>. Acesso em: 02 ago. 2026.

IBM. What is chain of thought (CoT) prompting?. 2026. Disponível em: <https://www.ibm.com/think/topics/chain-of-thoughts>. Acesso em: 02 ago. 2026.

KONISHI, Hidekazu. Claude Code features and settings reference 2026. 2026. Disponível em: https://hidekazu-konishi.com/entry/claude_code_features_settings_reference_2026.html. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: building an end-to-end agentic software development lifecycle with Azure and GitHub. 2026. Disponível em: <https://techcommunity.microsoft.com/blog/appsonazureblog/an-ai-led-sdlc-building-an-end-to-end-agentic-software-development-lifecycle-wit/4491896>. Acesso em: 02 ago. 2026.

MINDSTUDIO. What is an agent harness? The architecture behind Claude Code, Co-dex, and Cursor. 2026. Disponível em: <https://www.mindstudio.ai/blog/what-is-agent-harness-architecture-explained>. Acesso em: 02 ago. 2026.

OWASP FOUNDATION. MCP tool poisoning. 2026. Disponível em: https://owasp.org/www-community/attacks/MCP_Tool_Poisoning. Acesso em: 02 ago. 2026.

PILLITTERI, Pasquale. Claude Code harness: the runtime architecture that turns an LLM into an autonomous agent. 2026. Disponível em: <https://pasqualepillitteri.it/en/news/1892/claude-code-harness-runtime-architecture-2026-guide>. Acesso em: 02 ago. 2026.

PROMPTHUB. Prompt engineering for AI agents. 2026. Disponível em: <https://www.prompthub.us/blog/prompt-engineering-for-ai-agents>. Acesso em: 02 ago. 2026.

RESEARCHGATE. AI-first software development lifecycle: an agent-driven framework for autonomous planning, coding, testing, and deployment. 2026. Disponível em: https://www.researchgate.net/publication/403670772_AI-First_Software_Development_Lifecycle_An_Agent-Driven_Framework_for_Autonomous_Planning_Coding_Testing_and_Deployment. Acesso em: 02 ago. 2026.

WAVESPEED AI. Claude Code agent harness: architecture breakdown. 2026. Disponível em: <https://wavespeed.ai/blog/posts/claude-code-agent-harness-architecture/>. Acesso em: 02 ago. 2026.

WIKIPEDIA. Model context protocol. 2026. Disponível em: https://en.wikipedia.org/wiki/Model_Context_Protocol. Acesso em: 02 ago. 2026.